LECTURE 7: RECOMMENDER SYSTEMS Exercises

DS 8001

Ex1: Content-Based RecSys

Imagine you have a dataset containing information about various movies. Each movie has features such as genre, director, main actors and a brief description, and each user has a list of the movies they have rated.

We want a content-based recommender system that suggests new movies to a user based on the movies the user rated highly. A user has just joined and rated three movies: “Inception” (5 stars), “The Dark Knight” (4 stars) and “Interstellar” (5 stars). The five candidate movies below are unseen by this user. Use only the features in the table (the genre tags as listed and one main actor per movie):

Movie	Genre tags	Director	Main actor
Inception	Sci-Fi, Thriller	Christopher Nolan	Leonardo DiCaprio
The Dark Knight	Action, Crime	Christopher Nolan	Christian Bale
Interstellar	Sci-Fi, Drama	Christopher Nolan	Matthew McConaughey
Tenet	Sci-Fi, Action, Thriller	Christopher Nolan	John David Washington
The Prestige	Drama, Thriller	Christopher Nolan	Christian Bale
The Matrix	Sci-Fi, Action	Lana and Lilly Wachowski	Keanu Reeves
Gravity	Sci-Fi, Drama, Thriller	Alfonso Cuarón	Sandra Bullock
Titanic	Drama, Romance	James Cameron	Leonardo DiCaprio

Ex1: Content-Based RecSys - continued

Tasks:

(a) Define an item profile for each of the eight movies based on genre, director and main actor: a binary vector with one coordinate per feature value (each genre tag, each director – the Wachowskis count as one – and each actor), equal to 1 if the movie has that feature and 0 otherwise. How many coordinates does a profile have?

(b) Construct the user profile x as the weighted average of the item profiles i_j of the three rated movies, weighting each movie j by its rating r_j: $x = \sum_j r_j\, i_j \big/ \sum_j r_j$.

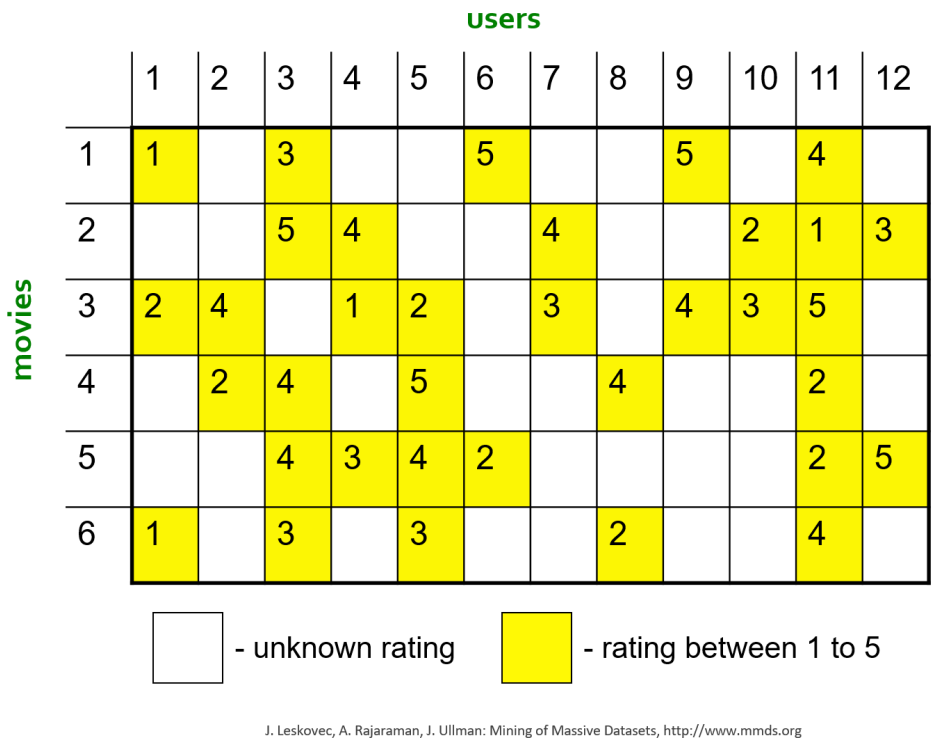
(c) Using cosine similarity, $\cos(x, i) = x \cdot i \,/\, (\|x\|\,\|i\|)$, score the five candidate movies and list the top 3 movie recommendations for this user.

(d) Which feature has the largest weight in x, and why? Which weakness of content-based recommendation does your top 3 illustrate? Could this method recommend anything to a user who has not rated any movie? How could a hybrid system help?

Toronto Metropolitan University

Ex2: Collaborative Filtering

Consider the user-item rating matrix below, from a movie recommendation system (it is the matrix of the lecture's item-item example). Each column represents a user, each row represents a movie, and each yellow cell contains a rating (1–5 stars); a cell is blank if the user hasn't rated the movie.



J. Leskovec, A. Rajaraman, J. Ullman: Mining of Massive Datasets, http://www.mmds.org

For User 1, use user-user collaborative filtering to predict their rating for Movie 4, which they haven't rated yet.

Ex2: Collaborative Filtering - continued

Measure the similarity of users x and y with the Pearson correlation coefficient from the lecture,

$$sim(x,y) = \frac{\sum_{s \in S_{xy}} (r_{xs} - \bar{r}_x)(r_{ys} - \bar{r}_y)}{\sqrt{\sum_{s \in S_{xy}} (r_{xs} - \bar{r}_x)^2} \sqrt{\sum_{s \in S_{xy}} (r_{ys} - \bar{r}_y)^2}},$$

where r_{xs} is the rating of user x for movie s, S_{xy} is the set of movies rated by both x and y, and $\bar{r}_x$ is the average of *all* the ratings of user x (not only those in S_{xy}).

(a) Calculate the similarity between User 1 and each user who has rated Movie 4.

(b) Identify the top 3 users most similar to User 1 (the neighborhood N, $k = |N| = 3$).

(c) Predict User 1's rating for Movie 4 using the weighted average of the ratings given to Movie 4 by these top 3 similar users, weighted by their similarities:
$\hat{r}_{xi} = \sum_{y \in N} s_{xy}\, r_{yi} \big/ \sum_{y \in N} s_{xy}$, with $s_{xy} = sim(x,y)$.

(d) How many movies does each user in N have in common with User 1? Show that if S_{xy} contains a single movie, then $sim(x,y)$ is $+1$ or -1 (or undefined). How much do you trust your prediction, and which drawback of collaborative filtering does this illustrate?

Ex3: Item-Item CF with Baselines

Five users (A–E) have rated four movies ($M1$–$M4$) on a 1–5 star scale; "–" means unknown. Rows are users and columns are movies, as in the lecture's utility matrix:

	$M1$	$M2$	$M3$	$M4$
A	–	3	3	2
B	3	1	–	2
C	5	3	2	4
D	4	–	4	–
E	–	1	3	2

We want to predict $\hat{r}_{E,M1}$, user E's rating of movie $M1$, with item-item collaborative filtering and a neighborhood of $k = 2$ movies.

The similarity s_{ij} of movies i and j is the *centered cosine*: subtract from each column the mean of its known ratings, set the unknown entries to 0, and take the cosine of the two resulting columns.

Ex3: Item-Item CF with Baselines - continued

(a) Compute $s_{M1,M2}$, $s_{M1,M3}$ and $s_{M1,M4}$.

(b) Find $N(M1;E)$, the $k = 2$ movies rated by E that are most similar to $M1$, and predict $\hat{r}_{E,M1} = \sum_{j \in N(M1;E)} s_{M1,j}\, r_{E,j} \big/ \sum_{j \in N(M1;E)} s_{M1,j}$.

(c) Now use the baseline estimate $b_{xi} = \mu + b_x + b_i$, where μ is the mean of all 15 known ratings, $b_x =$ (mean rating of user x) $- \mu$ and $b_i =$ (mean rating of movie i) $- \mu$. Predict

$$\hat{r}_{E,M1} = b_{E,M1} + \frac{\sum_{j \in N(M1;E)} s_{M1,j}\,(r_{E,j} - b_{E,j})}{\sum_{j \in N(M1;E)} s_{M1,j}}$$

and explain why the result differs from (b).

(d) On Netflix there are $|X| \approx 480{,}000$ users and $|S| = 17{,}770$ movies. Taking one similarity evaluation as the elementary operation, how many operations does it take to precompute the similarities of all pairs of users, and of all pairs of movies? By roughly what factor do the two counts differ? Give one more reason why item-item CF is often preferred in practice.

Ex4: Evaluating Recommendations

Five of a user's ratings were hidden before training (a held-out set). Two recommenders, A and B, were trained on the remaining data; their predictions for the five hidden ratings are:

Movie	$M1$	$M2$	$M3$	$M4$	$M5$
True rating r_{xi}	3	5	1	4	2
Prediction of A	4	3.5	2.5	3	2
Prediction of B	1	5	3.5	4.5	4

A movie is *relevant* to the user if its true rating is at least 4.

Ex4: Evaluating Recommendations - continued

(a) Compute the RMSE of each recommender on the five held-out ratings, $RMSE = \sqrt{\frac{1}{|R|}\sum_{(x,i)\in R}(\hat{r}_{xi} - r_{xi})^2}$, where R is the held-out set. Which recommender is more accurate by RMSE?

(b) Each recommender shows the user its top-k of these five movies, ranked by predicted rating. Compute precision@k = (number of relevant movies among the top k)/k for both recommenders, for $k = 2$ and $k = 3$.

(c) Which recommender would you deploy for a "Top 2 for you" row, and why? Relate your answer to the lecture's "Problems with Error Measures".

(d) The Netflix Prize required a 10% improvement in RMSE over Cinematch, which scored 0.9514 on the quiz set. What RMSE is 10% lower? By what percentage must the sum of squared errors (SSE) on the same quiz set fall?

Ex5: Learned Interpolation Weights

Recall the model with learned interpolation weights,

$$\hat{r}_{xi} = b_{xi} + \sum_{j \in N(i;x)} w_{ij}\,(r_{xj} - b_{xj}), \qquad J(w) = \sum_{(x,i) \in R} \left(\hat{r}_{xi} - r_{xi}\right)^2,$$

where b_{xi} is the baseline estimate of r_{xi} and the weights w minimize the training error $J(w)$.

Fix a target movie i with two neighbors j and k. Four users $x = 1, 2, 3, 4$ rated all three movies, $N(i;x) = \{j,k\}$ for each of them, and no other user rated i, so w_{ij} and w_{ik} appear in exactly four terms of J. Write $d_{xm} = r_{xm} - b_{xm}$ for the deviation of a rating from its baseline:

User x	d_{xj}	d_{xk}	d_{xi}
1	2	1	2
2	1	1	2
3	−1	−1	−1
4	−1	1	0.5

EX5: INTERPOLATION WEIGHTS - CONTINUED

(a) Show that $\hat{r}_{xi} - r_{xi} = w_{ij}\, d_{xj} + w_{ik}\, d_{xk} - d_{xi}$, and write the four terms of J that involve w_{ij}, w_{ik} as a function $J(w_{ij}, w_{ik})$. Evaluate it at the similarity weights $w_{ij} = 0.6$, $w_{ik} = 0.4$ (normalized to sum to 1, as in a similarity-weighted average).

(b) Using the lecture's gradient

$$\frac{\partial J}{\partial w_{ij}} = 2 \sum_{x} \left(\left[b_{xi} + \sum_{m \in N(i;x)} w_{im} (r_{xm} - b_{xm}) \right] - r_{xi} \right) (r_{xj} - b_{xj}),$$

compute ∇J at $w = (w_{ij}, w_{ik}) = (0, 0)$ and take one gradient-descent step $w \leftarrow w - \eta \nabla J$ with $\eta = 0.05$. Evaluate J before and after the step.

(c) Find the minimizer $w^* = (w_{ij}^*, w_{ik}^*)$ by setting both partial derivatives to zero. Compute $J(w^*)$ and compare it with (a) and (b). Do the learned weights sum to 1?

(d) Suppose instead that movie k is an exact duplicate of j ($d_{xk} = d_{xj}$ for every user x). Show that J then depends on w_{ij} and w_{ik} only through $w_{ij} + w_{ik}$. How do learned weights treat the duplicate, compared with similarity weights?

Ex6: Latent Factor Interpretation

Suppose a latent factor model with two factors has been trained, and the two factors are interpreted as the genres Comedy and Action. After training, you have the following factor vectors for users and movies:

- User factors p_x (Comedy, Action): U1 (0.1, 0.9), U2 (0.8, 0.2)
- Movie factors q_i (Comedy, Action): M1 (0.9, 0.1), M2 (0.2, 0.8), M3 (0.5, 0.5)

The model predicts $\hat{r}_{xi} = q_i \cdot p_x$. It has no bias terms, so these scores rank the movies for a user but are not on a star scale.

(a) For each user, compute the predicted score of each movie (show the dot product calculations), and predict which movie each user is likely to rate highest.

(b) A second training run on the same data returns rotated factors $p'_x = p_x O$ and $q'_i = q_i O$ (row vectors), where

$$O = \begin{bmatrix} 0.6 & -0.8 \\ 0.8 & 0.6 \end{bmatrix}, \qquad O\,O^T = I.$$

Compute p'_{U1}, q'_{M2} and $q'_{M2} \cdot p'_{U1}$. Show that every prediction is unchanged. What does this imply about naming the factors "Comedy" and "Action"?

Ex7: Matrix Factorization Optimization

Given a small user-item rating matrix ("?" = unknown):

	$M1$	$M2$	$M3$
$U1$	4	?	2
$U2$	3	3	?
$U3$	?	1	4

We fit a latent factor model with $k = 2$ factors, $\hat{r}_{xi} = q_i \cdot p_x$, where p_x (row x of P) is user x's factor vector and q_i (row i of Q) is movie i's. The matrices P and Q are initialized with small random numbers; use the values

$$P_0 = \begin{bmatrix} 0.3 & 0.1 \\ 0.2 & 0.2 \\ 0.1 & 0.4 \end{bmatrix} \begin{matrix} \leftarrow U1 \\ \leftarrow U2 \\ \leftarrow U3 \end{matrix} \qquad Q_0 = \begin{bmatrix} 0.4 & 0.1 \\ 0.2 & 0.3 \\ 0.1 & 0.5 \end{bmatrix} \begin{matrix} \leftarrow M1 \\ \leftarrow M2 \\ \leftarrow M3 \end{matrix}$$

and minimize the regularized objective from the lecture, where R is the set of known ratings:

$$\min_{P,Q} \sum_{(x,i)\in R} (r_{xi} - q_i \cdot p_x)^2 + \lambda_1 \sum_x \|p_x\|^2 + \lambda_2 \sum_i \|q_i\|^2.$$

Ex7: MF Optimization - continued

Use the learning rate $\eta = 0.01$ and the regularization parameters $\lambda_1 = \lambda_2 = 0.1$. One iteration of gradient descent computes, from the current P and Q,

$$\nabla p_{xf} = \sum_{i:\,(x,i)\in R} -2(r_{xi} - q_i \cdot p_x)\, q_{if} + 2\lambda_1 p_{xf},$$

$$\nabla q_{if} = \sum_{x:\,(x,i)\in R} -2(r_{xi} - q_i \cdot p_x)\, p_{xf} + 2\lambda_2 q_{if},$$

for every factor f, and then updates $P \leftarrow P - \eta \nabla P$ and $Q \leftarrow Q - \eta \nabla Q$.

(a) Which ratings appear in the objective? A classmate proposes to fill in the three missing ratings first – with the average known rating ($17/6 \approx 2.83$), or with 0 as plain SVD would – and then fit all nine entries. Explain why the objective sums over the known ratings only.

(b) Perform one iteration of gradient descent: compute the six errors $e_{xi} = r_{xi} - q_i \cdot p_x$, the gradients, and the updated matrices P_1 and Q_1.

EX7: MF OPTIMIZATION - CONTINUED

(c) Compute the effect of regularization on the first row p_{U1} of P: repeat its update with $\lambda_1 = 0.5$ (all else unchanged) and compare with (b). Show that the update can be written as $p_{U1} \leftarrow (1 - 2\eta\lambda_1)\, p_{U1} + 2\eta \sum_i e_{U1,i}\, q_i$, and explain how increasing λ_1 affects the values in P, after one iteration and after many. Does λ_1 change Q_1?

(d) Stochastic gradient descent (SGD) instead makes a step after each single rating r_{xi}. With the lecture's update rule (learning rate η; both updates computed from the old q_i and p_x),

$$\begin{aligned} \xi_{xi} &= 2(r_{xi} - q_i \cdot p_x), \\ q_i &\leftarrow q_i + \eta\,(\xi_{xi}\, p_x - \lambda_2\, q_i), \\ p_x &\leftarrow p_x + \eta\,(\xi_{xi}\, q_i - \lambda_1\, p_x), \end{aligned}$$

perform one SGD step on the rating $r_{U1,M1} = 4$, starting from P_0 and Q_0 (with $\eta = 0.01$ and $\lambda_1 = \lambda_2 = 0.1$).

How many known ratings does one gradient-descent iteration process, and how many does one SGD step process? With k factors, about how many arithmetic operations does each need? What does this mean on Netflix, with $|R| \approx 10^8$ known ratings?

Ex8: Predicting Ratings with LFMs

(a) Using the matrices P_1 and Q_1 from Ex7(b) (with $\lambda_1 = \lambda_2 = 0.1$), predict the missing ratings for $M2$ for user $U1$ and for $M3$ for user $U2$. Compare them with the known ratings: what do they tell you about the model after one iteration?

(b) The true values of these two ratings are unknown. Discuss how you would evaluate the accuracy of the model's predictions.

(c) The lecture improves the model with biases: $\hat{r}_{xi} = \mu + b_x + b_i + q_i \cdot p_x$. Estimate μ, b_x and b_i from the six known ratings as in "CF: Common Practice" (μ: overall mean rating; b_x = (mean rating of user x) $- \mu$; b_i = (mean rating of movie i) $- \mu$), and compute the baseline part $\mu + b_x + b_i$ for the same two missing ratings. Why does the biased model start from much better predictions than (a), and what is left for the term $q_i \cdot p_x$ to learn?

Toronto Metropolitan University